7

Walidacja kodu aplikacji

7.1. Wprowadzenie

Testowanie aplikacji ma na celu wykrycie błędów (walidacja oprogramowania) oraz sprawdzenie, czy oprogramowanie realizuje funkcje, do których wykonywania zostało przeznaczone (weryfikacja oprogramowania). Testowanie można przeprowadzić w dowolnym momencie podczas pracy nad oprogramowaniem, ale bardzo często jest to ostatni etap tworzenia aplikacji internetowej.

Testowanie nie spowoduje wykrycia wszystkich błędów oprogramowania, natomiast może dostarczyć informacji o zgodności oprogramowania z oczekiwaniami użytkowników oraz o występowaniu określonych błędów.

Celem testowania jest:

- ocena jakości aplikacji,
- wykrycie błędów oprogramowania i ich opisanie,
- określenie sposobu poprawienia oprogramowania.

Po przeprowadzeniu testów powinien zostać przygotowany raport podsumowujący, w którym będzie można znaleźć podsumowanie wykonanych prac, wskazanie potencjalnych zagrożeń, określenie poziomu bezpieczeństwa, opis proponowanych środków zaradczych.

Przeprowadzaniem testów całościowych aplikacji zajmują się najczęściej wyspecjalizowane firmy posiadające odpowiednie narzędzia.

7.2. Testy aplikacji

7.2.1. Podział testów

Jest wiele sposobów podziału testów. Testy można podzielić na przykład ze względu na:

- sposób wykonywania,
- zakres,
- funkcjonalność.

Ze względu na sposób wykonywania

Ze względu na sposób wykonywania testy dzielimy na:

- automatyczne,
- manualne.

Testy automatyczne

Podczas testowania oprogramowania często mamy do czynienia z sytuacją, w której czynności związane z testowaniem wykonywane są automatycznie. Testy automatyczne często są stosowane w testach wydajnościowych aplikacji. Zaletą takich testów jest to, że raz napisane mogą być wykorzystywane wielokrotnie. Ponadto testy zautomatyzowane można uruchamiać regularnie o określonych porach lub na pewnych etapach tworzenia aplikacji. Na przykład można ustawić testy tak, żeby były wykonywane każdej nocy i by rano był dostępny raport.

Testy automatyczne najczęściej opierają się na specjalnie do tego celu przeznaczonym oprogramowaniu.

Testy manualne

Testy manualne wymagają indywidualnego podejścia do testowanej aplikacji. Najczęściej są to testy dotyczące zgodności aplikacji z wizją zamawiającego. Każdorazowo są przygotowywane pod konkretne wymagania. Są wykonywane przez testera, który na podstawie scenariuszy testowych weryfikuje oprogramowanie.

Ze względu na zakres

Ze względu na zakres testy dzielimy na:

- jednostkowe,
- systemowe,
- integracyjne.

Testy jednostkowe

Testy jednostkowe (ang. *unit tests*) są testami automatycznymi, które porównują oczekiwany wynik funkcji z rzeczywistym rezultatem. Może to być test, w którym krok po kroku opisane są czynności, jakie ma wykonać program, i w którym jest określone, jaki powinien być rezultat jego działania. Do przeprowadzenia takiego testu może zostać wykorzystany program komputerowy, który wczytuje do aplikacji odpowiednie dane wejściowe, a następnie sprawdza, czy program generuje oczekiwane wyniki. Na przykład przy sprawdzaniu funkcji sumującej dwie liczby całkowite można w łatwy sposób ustalić dane wejściowe oraz oczekiwany wynik.

Testy jednostkowe często są nazywane testami czarnej skrzynki, ponieważ tester nie musi znać kodu zawartego w badanej funkcji. Podaje dane wejściowe i sprawdza, czy otrzymuje oczekiwane wyniki.

Testy systemowe

Testy systemowe przeprowadza się w celu weryfikacji aplikacji pod kątem zgodności z wymaganiami funkcjonalnymi oraz wymaganiami dotyczącymi wydajności, użyteczności i niezawodności. Testy systemowe sprawdzają, czy zintegrowany system spełnia jako całość wymagania zawarte w specyfikacji.

Testy systemowe są pierwszym poziomem, na którym system jest testowany jako całość, i są przeprowadzane w środowisku zbliżonym do rzeczywistych warunków, w jakich będzie działał system.

Testy integracyjne

Testy integracyjne służą do sprawdzania systemu na niższych poziomach. W takich testach weryfikowane są poszczególne komponenty systemu oraz interfejsy komunikacji między nimi. Testy integracyjne są przeprowadzane w celu określenia zgodności tych komponentów z wymaganiami funkcjonalnymi.

Ze względu na funkcjonalność

Ze względu na funkcjonalność testy dzielimy na:

- funkcjonalne,
- niefunkcjonalne.

Testy funkcjonalne

Testy funkcjonalne oceniają zgodność ze standardami, analizują przyczyny opóźnień w wyświetlaniu się różnych elementów strony, sprawdzają dostępność strony dla wyszukiwarek internetowych, weryfikują poprawność wyświetlanych komunikatów i komunikacji aplikacji z serwerem.

Testy niefunkcjonalne

Testy niefunkcjonalne sprawdzają cechy jakościowe systemu. Do testów niefunkcjonalnych należą:

Testy użyteczności (ergonomia) — oceniają przejrzystość treści, strukturę aplikacji, a także nawigację i estetykę aplikacji.

Testy kompatybilności (architektura) — sprawdzają skalowalność aplikacji i bazy danych, oceniają architekturę serwera i jego konfigurację oraz architekturę aplikacji.

Testy wydajnościowe — szukają wąskich gardeł aplikacji webowej, analizują strukturę bazy danych i zapytań do bazy, oceniają kod źródłowy aplikacji. Wydajność jest jednym z podstawowych aspektów funkcjonowania aplikacji internetowej. Z aplikacji najczęściej korzysta wielu użytkowników — testy wydajnościowe mogą pokazać, czy aplikacja przy określonym obciążeniu będzie działała prawidłowo.

Testy bezpieczeństwa — sprawdzają, jakie zabezpieczenia przed atakami wykorzystuje aplikacja, w jakim stopniu działają elementy zabezpieczające systemu, w jakim stopniu próby ataków są wykrywane.

Inne rodzaje testów

Testy z udziałem użytkownika

Testy z udziałem użytkownika są przeprowadzane, gdy w aplikacjach występują obszary, które są istotne z punktu widzenia użytkownika. Określają one, jak użytkownik wchodzi w interakcję z aplikacją, aby uzyskać oczekiwany wynik.

Testy akceptacyjne

Testy akceptacyjne są przeprowadzane w celu potwierdzenia, że wykonane oprogramowanie jest odpowiedniej jakości. Testy te sprawdzają, czy aplikacja jest zgodna z projektem i założeniami. Czy zawiera wszystkie przewidziane elementy i czy jej wygląd nie odbiega od oczekiwanego.

7.3. Debugowanie aplikacji

Debugowanie aplikacji to proces wyszukiwania błędów w programie i ich usuwania.

Analizowanie kodu tworzonej aplikacji internetowej w celu znalezienia błędów powinno być podstawowym działaniem programisty. Coraz większa liczba frameworków używanych do tworzenia stron wymaga analizy nie tylko kodu strony, ale również zwracanego kodu wynikowego aplikacji internetowej.

Analizę kodu można przeprowadzić:

- na poziomie kodu źródłowego w sposób zautomatyzowany,
- w czasie wykonania programu, często również w sposób zautomatyzowany,
- korzystając z pomocy programisty, który zlokalizuje błędy oraz zaproponuje poprawki.

Analiza kodu powinna być wykonywana ręcznie przez programistę, a następnie należy przeprowadzić analizę automatyczną dotyczącą między innymi przepływu sterowania, zmian w drzewie DOM, wartości zmiennych. Trzeba także sprawdzić, jak zachowuje się przeglądarka w trakcie wykonywania określonych działań aplikacji.

7.3.1. Narzędzia do debugowania

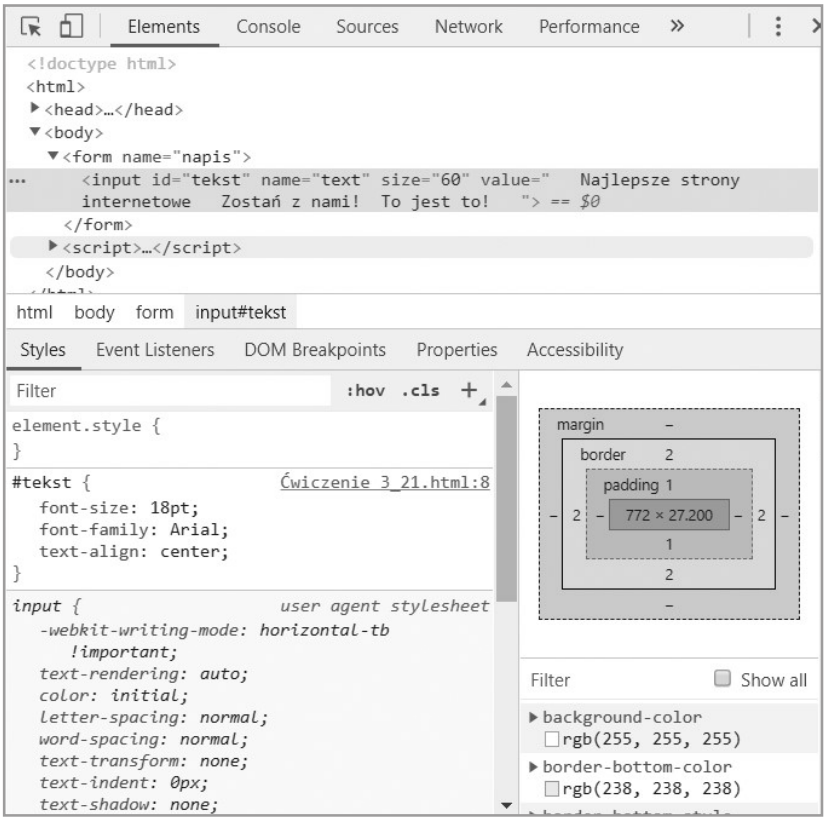
Podstawowe narzędzia potrzebne do edycji i debugowania kodu aplikacji znajdują się w przeglądarkach.

Na przykład dla przeglądarki Firefox dostępny jest dodatek Firebug oraz zestaw narzędzi Firefox Developer Tools, a przeglądarka Google Chrome ma zestaw narzędzi Chrome Developer Tools. Te i inne programy narzędziowe są do siebie bardzo podobne i udostępniają analogiczne funkcje.

Chrome Developer Tools

Chrome Developer Tools to zestaw narzędzi dla programistów aplikacji internetowych. Jest wbudowany bezpośrednio w przeglądarkę Google Chrome. Można go uruchomić, klikając element strony prawym przyciskiem myszy i wybierając opcję *Zbadaj*. Inna metoda to wybranie klawiszy *Ctrl+Shift+I* lub klawisza *F12*.

Po uruchomieniu narzędzia zostanie otwarte okno Chrome Developer Tools. W górnej części okna znajduje się lista paneli, niżej dostępny jest obszar roboczy wybranego panelu (rysunek 7.1).



Rysunek 7.1. Okno Chrome Developer Tools. Panel Elements

Panel Elements

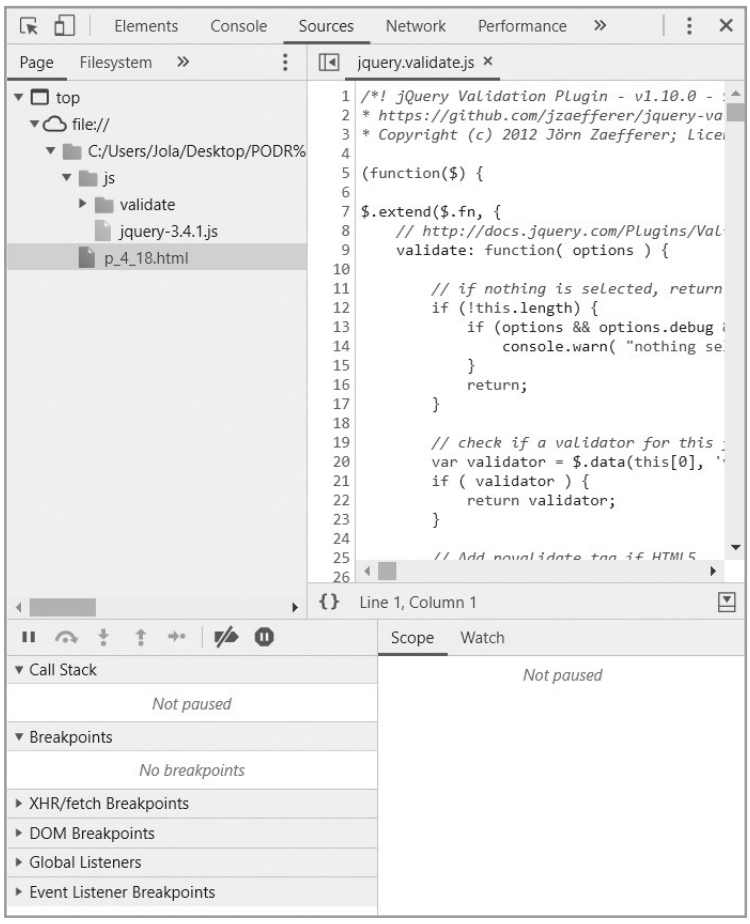
Panel *Elements* służy do edycji wyświetlanej strony. Pokazuje drzewo DOM strony i pozwala je modyfikować. W panelu można zobaczyć, z jakich elementów składa się strona. Można przeprowadzić edycję tych elementów i stylów, które są do nich przypisane.

Element strony zostanie wybrany po jego kliknięciu w strukturze HTML. W dolnej części panelu zostaną pokazane style CSS przypisane do tego elementu. Z prawej strony zostanie wyświetlona informacja dotycząca wybranego węzła DOM. Są tu widoczne aktualne ustawienia elementów oraz lista właściwości przypisanych do elementu.

Panel Sources

W panelu *Sources* wyświetlane są wszystkie zasoby, z których korzysta strona. Można w nim podglądać skrypty wykonywane na stronie, wykonać ich edycję i debugowanie. Można dodać skrypty zapisane jako plik *.js* lub dołączyć kody dostępne przez różne rozszerzenia Google Chrome.

Okno panelu zostało podzielone na trzy części. W pierwszej części u góry po lewej stronie można wybrać określony skrypt, w drugiej części z prawej strony widoczna jest zawartość tego skryptu. W trzeciej części u dołu można zobaczyć typowe funkcje debugera (rysunek 7.2). Dostępne opcje pozwalają na szukanie błędów zarówno w skrypcie, jak i w drzewie DOM.



Rysunek 7.2. Okno panelu Sources

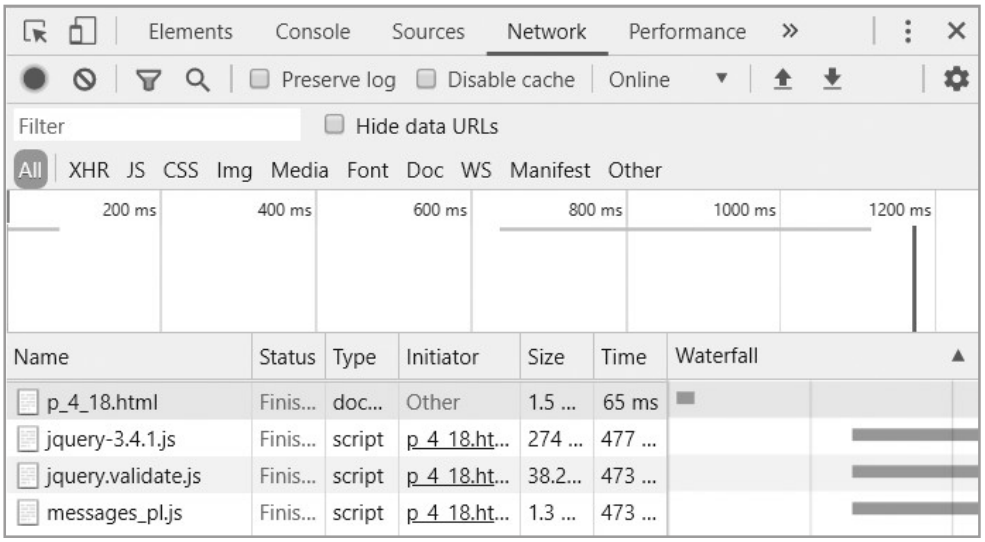
Panel Console

Panel *Console* służy do pisania kodu i przeglądania zarejestrowanych komunikatów, które informują, czy program działa zgodnie z oczekiwaniami.

Panel Network

Panel *Network* pozwala analizować ruch sieciowy na stronie. Można podglądać komunikację, filtrować ją i analizować czas odpowiedzi na żądanie.

Panel pokazuje pliki pobierane przez stronę. Może służyć do sprawdzenia, czy wszystkie elementy strony zostały załadowane prawidłowo. Umożliwia monitorowanie statusu asynchronicznych odwołań (rysunek 7.3).



Rysunek 7.3. Okno panelu Network

Panel Audits

Panel *Audits* pozwala na wykonanie statycznej i dynamicznej analizy kodu strony. Po przeprowadzeniu audytu narzędzie podpowiada, jakie elementy można zmienić, aby przyspieszyć działanie witryny.

Panel Performance

Panel *Performance* służy do symulowania pracy aplikacji w celu tworzenia optymalnego kodu i poprawienia jej wydajności.

7.4. Pytania i zadania

7.4.1. Pytania

1. Wymień rodzaje testów całościowych aplikacji internetowej.
2. Jak działają testy jednostkowe?
3. Jaki jest cel przeprowadzania testów wydajnościowych?
4. W jakim celu przeprowadza się testy funkcjonalne?
5. Na czym polega debugowanie aplikacji internetowej?
6. Jakie narzędzia służą do debugowania aplikacji internetowej?